

Full Scale – Build Doc v2

2026-09-19

Organised by ownership, not narrative. Five people pick a component and build against the contracts in §3–4.

Live editable copy: <https://claude.ai/code/artifact/d5148499-b861-4db6-a48b-0d17cdcda9b8>

Positioning

IKEA Kreativ already ships the v1 core loop, so dimensional accuracy is no longer the pitch. Kreativ does LiDAR room scan, 3D replica with real dimensions, AI erase of existing furniture, placement at accurate scale, and checkout. It is free, inside the IKEA app, and patented. Assume every judge has used it.

What is left, and it is the whole product: **every competitor is catalog-locked and we are not.**

CLAIM	WHY IT HOLDS	WHERE IT IS BUILT
Ingest an object that is not for sale	Kreativ places IKEA SKUs, Houzz places Houzz Shop SKUs, Roomvo places partner surfaces. None can take the desk from your parents' basement, the Marketplace couch, or the floor model you are standing next to. They are retailer tools, so there is nothing in it for them.	A + C, pipeline P2
Retrieval over your own possessions	Each scanned object becomes a document: image embedding, caption, material and colour palette, true dimensions. That supports "find something that fits the 80 cm gap beside my desk and matches its wood tone" – dense vector for style, numeric filter for fit, over your stuff plus catalog. Kreativ structurally cannot do this, because it never captures your belongings.	F, pipeline P4
Cross-catalog, not single-brand	One scene holds an IKEA-class product, a Shopify merchant product, and your own chair.	C + F, pipeline P3
True 1:1 in a headset	Stand inside the room at real scale, not a phone-sized window on it.	D, pipeline P6
Version history	Content-hashed layouts with parent pointers, scrubbed on a timeline.	B + D
Moving day	Will my stuff fit in the new place. The one case that is acute, recurring, and unserved.	E

Pitch order

1. Lead with **the catalog is an input, not the product**. Show an object that is for sale nowhere.
2. Then **retrieval over your possessions** — the query no retailer tool can answer.
3. Then the fit engine result, with a number attached.
4. Never open with "redesign your room" or with measurement accuracy. Kreativ owns both sentences.

What this changes from v1

v1 led with fit certainty against a field of "dimensionally a lie" tools. That line is now false against Kreativ, so it is retired. Fit certainty stays as a proof beat, not as the headline.

Ownership map

Six components, five people. One person owns C and E together, because both are server-side Python against the same fixtures.

#	COMPONENT	OWNER	EXPOSES TO EVERYONE ELSE	CRITICAL PATH
A	iOS capture + app shell (Expo)	TBD – must be the Swift person	RoomCapture v1 JSON, Object v1 with state:"measured" in under 1 s	Yes
B	Backend + data layer (Cloudflare)	TBD	The whole /v1 HTTP surface, R2 keys, the room SSE stream	Yes
C	3D generation (Baseten)	TBD (also owns E)	A GLB at R2 key objects/{id}/mesh.glb, already bound to bboxMeters	Yes
D	WebXR / Quest runtime	TBD	Nothing. D is a pure consumer of B.	Demo only
E	Fit engine + solver	Same owner as C	POST /v1/fit → FitReport v1, POST /v1/solve → placements	No, but it is the pitch
F	Retrieval + agents + voice	TBD	POST /v1/search, the OMNI voice loop on the phone	No

Rules that make this work

1. A component never reads another component's internals. It reads the contract in §3 and §4.
2. Every /v1 endpoint answers from a committed fixture when the request carries X-Stub: 1. B ships the stub layer in hours 0–2, before any real logic.
3. The mesh scale binding happens exactly once, in C. A, D, and E never rescale a GLB. If two components both rescale, the object is wrong by the square of the error and nobody finds it until the demo.
4. D earns zero sponsor points. There is no AR/VR track in 2026. Treat every headset hour as demo spend, not investment.

Staffing risk

If no one is comfortable in Swift, component A cannot produce real measurements. The product then becomes a visualiser with estimated numbers, which is the thing Kreativ already does better. Verify the Swift owner before H0, not at H4.

Contracts, part 1 – the four schemas

Write these four files before you write anything else. Everything in §5 and §6 is an implementation detail behind them.

Global conventions

These are not negotiable per component. A component that breaks one of them corrupts every other component silently.

THING	CONVENTION
Length	Metres, float. Never centimetres, never inches, never millimetres. Convert at the UI edge only.
Angle	Degrees, float, counter-clockwise seen from +Y.
Money	Integer cents plus an ISO 4217 code. Never a float.
Time	ISO 8601 with a <code>Z</code> suffix. Compute in UTC, convert once for display.
Axes	Right-handed, +Y up. The ARKit world frame with <code>.gravityAndHeading</code> , so <code>-Z</code> points to true north and <code>+X</code> points east.
Transform	16 floats, column-major. This matches <code>simd_float4x4</code> memory order and <code>THREE.Matrix4.fromArray</code> with no transpose.
Id	UUID v4 string, minted by the client, so a scan works offline and reconciles later.

RoomCapture v1 – A writes, B stores, D and E read

```
{
  "schemaVersion": 1,
  "roomId": "uuid",
  "capturedAt": "2026-09-20T02:11:04Z",
  "worldAlignment": "gravityAndHeading",
  "northBearingDeg": 0.0,
  "floor": { "polygon": [[x, z]], "areaM2": 18.4 },
  "walls": [{
    "id": "uuid",
    "transform": [16 floats],
    "dimensions": [width, height, thickness],
    "confidence": "high|medium|low"
  }],
  "openings": [{
    "id": "uuid",
    "kind": "door|window|opening",
    "wallId": "uuid",
    "transform": [16 floats],
    "dimensions": [width, height, 0],
    "hingeSide": "left|right|unknown",
    "swingDeg": 90
  }],
  "objects": [{
    "id": "uuid",
    "category": "table|chair|bed|sofa|storage|...",
    "transform": [16 floats],
    "dimensions": [w, h, d],
    "confidence": "high|medium|low"
  }]
}
```

Notes that matter:

- `worldAlignment` must be the literal `"gravityAndHeading"`. If A ever emits anything else, B rejects the upload with HTTP 422. Without true north the sun simulation is invented, and retrofitting means re-scanning every room.
- `northBearingDeg` is redundant with the frame, but store it anyway. It is the one value a reviewer can check against a compass on the table.
- RoomPlan already gives segmentation. Object removal is `object.visible = false` on a thing the API labelled. Nobody implements segmentation.
- Transport is JSON, never USDZ. D rebuilds walls as boxes, openings as gaps, and objects as per-category proxies. The USDZ → GLB conversion chain is where the weekend dies.

Object v1 – the convergence type

Every object, from a phone scan, a Shopify product, or a primitive fallback, is this shape. The fit engine reads `bboxMeters` and nothing else.

```
{
  "schemaVersion": 1,
  "objectId": "uuid",
  "source": "scan|catalog|primitive",
  "state": "measured|generating|ready|failed",
  "name": "MacBook Pro 14",
  "category": "laptop",
  "glbUrl": null,
  "bboxMeters": { "w": 0.3126, "h": 0.0155, "d": 0.2212 },
  "measure": { "method": "lidar|extracted|declared", "confidence": 0.94 },
  "caption": "matte space-grey aluminium laptop, closed",
  "palette": ["#3a3a3c", "#8e8e93"],
  "price": { "cents": 199900, "currency": "CAD" },
  "productUrl": null,
  "merchant": null,
  "createdAt": "2026-09-20T02:12:09Z"
}
```

`state` is the perceived-latency fix, and it lives in the schema rather than in the UI. A returns `state:"measured"` with a real `bboxMeters` and a null `glbUrl` in under one second. The client renders the measured box with its numbers straight away. `glbUrl` arrives later, over SSE, when `state` becomes `ready`.

The mesh normalisation contract

C guarantees all four of these for every GLB it publishes. This is the single most important sentence in the document for anyone downstream.

1. The mesh axis-aligned bounding box equals `bboxMeters`, to within 1 mm.
2. The origin sits at the bottom-centre of that box, so a placement `y` of 0 means "on the floor".
3. +Y is up and -Z is the front face.

4. Units in the GLB are metres, so the glTF node scale is 1.

A consumer that applies its own scale factor is a bug, not a preference.

Placement v1 and Version v1 – B stores, D renders, E validates

```
{
  "placementId": "uuid",
  "objectId": "uuid",
  "p": [x, y, z],
  "yawDeg": 0.0,
  "scale": 1.0,
  "lockedToWallId": null,
  "flags": ["blocks_door_swing"]
}
```

```
{
  "schemaVersion": 1,
  "versionId": "uuid",
  "roomId": "uuid",
  "parentId": "uuid|null",
  "label": "under $1200",
  "createdAt": "2026-09-20T04:40:00Z",
  "placements": [],
  "materials": { "wall": "#8a9a7b", "floor": "oak-natural", "trim": "#ffffff" },
  "contentHash": "sha256 of placements + materials, keys sorted"
}
```

scale exists only for an explicit user override and must be 1.0 everywhere else. E raises a warning when it sees any other value, because a non-unit scale means somebody broke the normalisation contract upstream.

A version is immutable. An edit writes a new version with parentId set to the old one. That is what makes history free: hash the layout, keep parent pointers, diff two versions into added, removed, and moved.

Contracts, part 2 – API, storage, and stubs

B ships this surface as stubs in hours 0–2. Real logic lands behind it afterwards. Nobody waits.

HTTP surface (Cloudflare Workers, base /v1)

METHOD + PATH	BODY	RETURNS	CONSUMER
POST /rooms	RoomCapture v1	{ roomId }	A
GET /rooms/{id}	–	RoomCapture v1	D, E

METHOD + PATH	BODY	RETURNS	CONSUMER
POST /rooms/{id}/versions	Version without ids	Version v1	A, D, F
GET /rooms/{id}/versions	–	[{ versionId, label, createdAt, parentId }]	A, D
GET /versions/{id}	–	Version v1	D
POST /uploads	{ kind, ext }	{ key, putUrl } presigned R2	A, C
POST /objects	{ source, name, category, bboxMeters, measure, frameKeys[] }	Object v1 with state:"measured"	A
GET /objects/{id}	–	Object v1	all
POST /objects/{id}/generate	{ tier: "live" "quality" }	{ jobId }	A, C
GET /jobs/{id}	–	{ state, progressPct, objectId, error }	A
POST /search	{ text?, imageKey?, fit?, source?, limit }	[{ objectId, score, object }]	F
POST /fit	{ roomId, versionId } or { roomId, placements }	FitReport v1	A, D
POST /solve	{ roomId, intent, budgetCents?, fixed[] }	{ placements, objective, infeasible? }	F
POST /push/{roomId}	{ versionId }	204	A, F
GET /sync/{roomId}	–	SSE stream	D

`fit` in a search body is a numeric filter, not a vector term: `{ "maxW": 0.8, "maxH": 1.2, "maxD": 0.6 }`. That is the half of the query Kreativ cannot express.

FitReport v1

```
{
  "ok": false,
  "checkedAt": "2026-09-20T05:02:00Z",
  "violations": [{
    "kind": "door_swing|clearance|wall_gap|window_occlusion",
    "severity": "block|warn",
    "placementId": "uuid",
    "detailMeters": 0.11,
    "message": "blocks the door swing by 11 cm",
    "geometry": { "type": "arc", "center": [x, z], "radiusM": 0.9, "startDeg": 0, "endDeg": 90 }
  }]
}
```

`geometry` exists so D and A can draw the violation in red without re-deriving it. Supported types are `arc`, `polyline`, and `rect`. E owns the list. Adding a fifth type is a contract change, so announce it.

SSE stream, `GET /sync/{roomId}`

One Durable Object per room does the fan-out. Event payloads:

```
event: object    data: Object v1           // state changed, usually measured -> ready
event: version   data: { versionId }       // a new version was pushed
event: fit       data: FitReport v1        // a re-check finished
```

The Quest subscribes on room open and never polls. This is pipeline P6.

R2 key layout

KEY	WRITTEN BY
<code>rooms/{roomId}/capture.json</code>	B, on upload
<code>objects/{objectId}/frames/{n}.jpg</code>	A, presigned
<code>objects/{objectId}/mesh.glb</code>	C
<code>objects/{objectId}/thumb.jpg</code>	C
<code>catalog/{merchant}/{productId}/source.jpg</code>	P3 crawler
<code>fixtures/...</code>	committed by hand at H0

Keys are derived, never stored as URLs in D1. A URL in a row is a cache-invalidation bug waiting to happen.

D1 tables

```
rooms(id TEXT PK, name, captured_at, north_bearing_deg REAL, created_at)
versions(id TEXT PK, room_id, parent_id, label, content_hash,
         placements_json TEXT, materials_json TEXT, created_at)
objects(id TEXT PK, source, state, name, category,
        bbox_w REAL, bbox_h REAL, bbox_d REAL,
        measure_method, measure_confidence REAL,
        caption, palette_json, price_cents INTEGER, currency,
        product_url, merchant, created_at)
jobs(id TEXT PK, object_id, kind, tier, state, progress_pct, error, created_at, updated_at)
```

Vectorize index `objects-v1`

768 dimensions, cosine metric, from CLIP ViT-L/14 image embeddings. Filterable metadata, integers in millimetres so numeric range filters work:

```
objectId, source, category, w_mm, h_mm, d_mm, dominant_hex
```

That is pipeline P4 in one line: dense vector for style, integer range filter for fit.

The stub rule

Every endpoint returns a committed fixture when the request carries `X-Stub: 1`. Four fixtures go in the repository in hour 0, before any component compiles:

1. `fixtures/room-demo.json` — a real RoomPlan capture of any room, frozen.
2. `fixtures/object-macbook.json` — `state:"ready"`, real numbers.
3. `fixtures/mesh-macbook.glb` — already bound to those numbers.
4. `fixtures/fitreport-doorswing.json` — one blocking violation with an arc.

D can build the whole headset runtime against these with no phone and no server. That is the point.

Components A, B, C

A – iOS capture and app shell (Expo)

Scope. Expo Router, Expo UI, two custom native modules in Swift. One wraps `RoomCaptureView` for room scan. One wraps a RealityKit `ARView` for AR placement, with a USDZ plus QuickLook fallback. 3D browsing on the phone uses `expo-gl` and `three.js`. A Live Activity shows generation progress. Haptics on a fit violation.

Depends on. B's `/v1` surface only. It builds against `X-Stub: 1` from hour 0.

Exposes. `RoomCapture v1`, and `Object v1` with `state:"measured"` in under one second.

Done when. A judge scans a room and an object, both reach B, and the measured box renders with real numbers before the mesh exists.

Cut if behind, in order.

1. The RealityKit AR module. Fall back to USDZ and QuickLook, which is 30 minutes of work.
2. The Live Activity. A progress bar in the app says the same thing.
3. The on-phone `three.js` browser. The headset already shows the scene.
4. Last resort: drop Expo entirely, ship pure Swift, and lose the primary track.

Traps.

- `expo-dev-client`, `npx expo prebuild`, and a local Xcode build. Expo Go cannot load a custom native module.
- A config plugin for `NSCameraUsageDescription`, or the app crashes on first camera use.
- LiDAR device required: iPhone 12 Pro or later, iOS 16 or later.
- A free Apple developer account works, with a 7-day provisioning profile.
- Time-box the RoomPlan module to 4 hours. See the H4 gate in §9.

A custom native module wrapping an Apple framework is the Expo differentiator. Most Expo entries never leave the JavaScript sandbox.

B – Backend and data layer (Cloudflare)

Scope. Workers for orchestration, R2 for GLBs and frames, D1 for rooms, versions, objects, and jobs, Vectorize for embeddings, a queue for generation jobs, and one Durable Object per room for the SSE fan-out.

Depends on. Nothing. B is the only component that can start at minute zero with no blockers, which is why B owes everyone else the stub layer first.

Exposes. Everything in §4.

Done when. Every endpoint in §4 answers with the right shape, live or stubbed, and the room SSE stream delivers an `object` event end to end.

Cut if behind, in order.

1. Vectorize. Fall back to a brute-force cosine scan over the objects table. At a few hundred objects that is fast enough, and it keeps F alive.
 2. The job queue. Call Baseten inline with a longer timeout.
 3. Auth. A device id header is enough for a weekend.
- Order of work.** Stub layer, then `/objects` and `/uploads`, then `/rooms`, then SSE, then the rest. The order matches the critical path in §8.

C – 3D generation (Baseten)

Scope. Background removal, image-to-3D on Baseten, and the scale binding. Two latency tiers behind one `tier` parameter.

TIER	MODEL	BEHAVIOUR	USE
live	Stable Fast 3D	Sub-second on an A100, about 6 GB VRAM, MIT licence, UV unwrap and PBR parameters	The on-stage generation
quality	TRELLIS 2 or Hunyuan3D Pro	Slower, better	Pre-baked catalog, async upgrade

Depends on. B for job records and R2 keys. Nothing else.

Exposes. A GLB at `objects/{id}/mesh.glb` that satisfies all four clauses of the mesh normalisation contract in §3.

Done when. An arbitrary object photographed on the venue floor returns a GLB whose bounding box matches the LiDAR measurement to within 1 mm, and a tape measure on the table agrees.

The binding is the technical thesis. Generated meshes come out at normalised scale, roughly -1 to 1 per axis. Rescale the generated box to the LiDAR box. It is about ten lines of code, and it is the entire reason the AR view is true rather than decorative. Say it to a technical judge in one sentence: the generative model gives shape, the depth sensor gives size, and we bind them.

Cut if behind, in order.

1. The `quality` tier. Run everything on Stable Fast 3D.
2. Background removal. Shoot against a clean surface instead.
3. Generation itself. Fall back to a textured box at the measured dimensions. Ugly, but still dimensionally true, and the fit engine does not care.

Known failure set. Single-image generation degrades badly on transparent, reflective, thin, and very dark objects. A MacBook is a good demo object, because it is matte, rectangular, and solid. A water bottle is the adversarial case. Test twenty random objects on Friday so the boundary is known rather than discovered on stage. Multi-view input beats single-image, and a sweep gives four good frames for free.

Components D, E, F

D – WebXR / Quest runtime

Scope. three.js with `@react-three/xr`. Rebuild the room from `RoomCapture v1` primitives: walls are boxes, openings are gaps, unscanned furniture is a per-category proxy. Load GLBs by URL. Grab, move, and rotate an object. A version scrubber on the timeline. Draw `FitReport` geometry in red.

Depends on. B's `GET /rooms/{id}`, `GET /versions/{id}`, and the SSE stream. It can run the entire build against the four fixtures with no phone and no server.

Exposes. Nothing. D is a pure consumer, which is exactly why it is safe to staff with whoever is least available.

Done when. A person in the headset stands in the scanned room at 1:1, an object pushed from the phone appears within 3 seconds, and the version scrubber replays history.

Cut if behind, in order.

1. The version scrubber in the headset. Keep it on the phone, where the Expo judges see it.
2. Object manipulation. A read-only walkthrough still lands the 1:1 beat.
3. The whole headset. See §10 for the phone-only fallback.

Do not use Unity. It costs about five hours in build-and-deploy cycles. WebXR has no build step, hot reloads from a laptop, and renders the same scene in a desktop browser for the casting monitor.

Remember the economics. There is no AR/VR track in 2026. Ubisoft ran one in 2024, Snap in 2025. Every hour here earns zero sponsor points.

E – Fit engine and solver

Scope. Two separate things behind two endpoints.

`POST /fit` is a validator. It checks a layout and reports violations:

- **Clearance corridors.** Walking paths between placed objects against a minimum. 90 cm is a good default and sits in the AODA and ADA neighbourhood.
- **Door swing arcs.** RoomPlan gives door position and width. Sweep the arc and intersect it with placed geometry.
- **Window occlusion.** Does this block the light.
- **Wall adjacency.** Is it against the wall, or floating 4 cm off it.

`POST /solve` is an optimiser. Discretise the floor to a grid and run OR-Tools or `scipy.optimize`. An LLM translates "cozy reading corner" or "redo this under \$1,200" into an objective function and constraints. The solver places things.

Never let an LLM emit coordinates. The 2025 Shopify track winner paired camera checkout with a real MINLP procurement optimiser. Those judges reward actual optimisation under a legible demo.

Depends on. `RoomCapture v1` and `bboxMeters`. Nothing else. E never loads a mesh.

Exposes. `FitReport v1` and solved placements.

Done when. The solver returns infeasible on an over-stuffed room and the system says so out loud: there is no arrangement of these three pieces that keeps a 90 cm walkway to the door; drop the ottoman, or go 20 cm narrower on the couch. A system that can be wrong and knows it beats a demo made of vibes.

Cut if behind, in order.

1. The solver. Keep the validator. A warning is 80% of the beat for 20% of the work.
2. Window occlusion and wall adjacency. Keep door swing and clearance.
3. Sun simulation, which is an optional extension of window occlusion. If it ships, compute solar position from latitude, longitude, and timestamp with the NOAA algorithm, about 40 lines and no API key. Never fake it: `worldAlignment` already gives real compass bearings, so faking costs credibility for zero saving.

F – Retrieval, agents, and voice

Scope. Three pieces that share one owner.

1. **Indexing.** Every object gets an image embedding, a caption, a colour palette, and its true dimensions, written to `Vectorize` on `state:"ready"`.
2. **Hybrid search.** Dense vector for style, integer range filter for fit, over your own objects plus catalog. This is the differentiator from §1, so it outranks the agent loop if time is short.
3. **Voice (OMNI).** One unbroken loop on the phone: hold the phone at an object, ask "what is this, will it fit beside my desk?", get an answer aloud from live video plus measured geometry.

The multi-agent design loop, scout → fit → style → budget, sits on top of search and solve. It is the second Huawei track and the openJiuwen angle.

Depends on. B's `/search` and `/solve`. Objects must reach `state:"ready"` first.

Exposes. The search endpoint behaviour and the voice loop.

Done when. "Find something that fits the 80 cm gap beside my desk and matches its wood tone" returns results that actually fit, and the dimension filter provably excludes an object that is 5 cm too wide.

Cut if behind, in order.

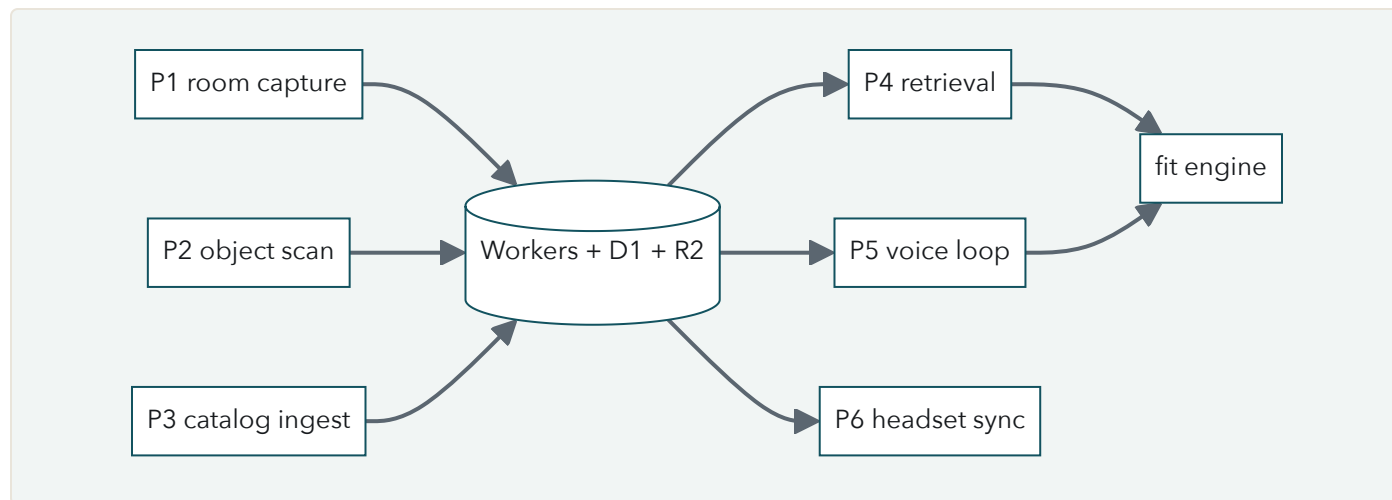
1. The multi-agent loop. One LLM call does the same job on stage.
2. In-headset voice. Quest browser speech support is inconsistent. If it must ship, use push-to-talk on the controller streaming audio to the server, and never trust the Web Speech API to be present.
3. Search itself, last. Losing it costs the second differentiator.

Build voice on the phone first. The native Speech framework is on-device, has no latency, and is completely reliable. The phone is already in the demo and already in someone's hand. Voice is not a garnish in a headset, because there is no keyboard and controller text entry is miserable.

Voice also buys three things: the speech modality that Huawei OMNI Live requires, a natural ElevenLabs integration for responses, and cover for generation latency. A narrating agent turns 15 seconds of dead air into 15 seconds of progress.

Pipelines P1-P6

A component is a person. A pipeline is a handoff between two people. Each pipeline below names its owner, its handoff, the one detail that makes it work, and the failure it owns.



P1 – Room capture

A → B → D. RoomPlan produces `CapturedRoom`, serialised as `RoomCapture v1` JSON, never USDZ. Upload, then rebuild in `three.js` from primitives.

The detail. Set `ARConfiguration.worldAlignment = .gravityAndHeading` on the capture session from day one. Without it the room has no absolute orientation, window normals are meaningless, and the sun comes from an arbitrary direction while looking completely plausible. Retrofitting means re-scanning everything.

Failure it owns. The USDZ → GLB conversion chain: coordinate conventions, unit scaling, material loss, headless Blender in a container at 3 am. JSON transport avoids the class entirely and produces a cleaner room than a photogrammetric mesh would.

P2 – Object scan

A → C → B → D. The LiDAR bounding box returns in under one second and renders straight away with real numbers. Frames go to Baseten. The generated mesh comes back at normalised scale, roughly -1 to 1 per axis. Rescale it to the LiDAR-measured box.

The detail. That binding is ten lines and it is the whole technical thesis. Metric-scale reconstruction from single-view RGB-D has no reliable general solution, which is why every consumer image-to-3D tool ships dimensionally meaningless output. We do not solve it. We sidestep it with a depth sensor.

Failure it owns. Perceived latency. The fix is architectural, not cosmetic: `state:"measured"` exists so the phone shows a true number while the mesh is still generating.

P3 – Catalog ingest

Most cuttable. Shopify storefronts expose `/products.json` and `/collections/<handle>/products.json` with no auth. Verify 15–25 furniture merchants before H0, because some merchants disable it.

Shopify has a weight field but no standard dimensions field, so dimensions live in metafields, free text in `body_html` in mixed units, variant titles like `60" x 30"`, spec images, or nowhere.

1. Regex pass on numbers next to width, depth, height, W, D, H tokens. About 60% for almost no work.
2. LLM pass over the remaining `body_html` with a constrained output schema.
3. VLM pass on spec images where there is no text at all.
4. Validation. Unit sanity, so a sofa is not 8 cm wide. Category priors, so a dining chair is 40–50 cm. Cross-check the claimed width-to-height ratio against the product photo aspect ratio.
5. Confidence score. Low confidence surfaces as "unverified fit" rather than a silent guess.

The detail. Step 4 is what the Rox rubric rewards, because it is genuine multi-source resolution. Step 5 is the difference between an agent and a scraper.

Failure it owns. Pre-bake 60–100 products offline and cache the GLBs. Do exactly one live generation on stage, on an object the judge names, with the voice agent narrating over the wait.

P4 – Retrieval

F Object → embedding, caption, palette, dimensions → Vectorize → hybrid query → results into the scene.

The detail. The query has two halves that must not be mixed. Style is a dense vector similarity. Fit is an integer range filter on `w_mm`, `h_mm`, `d_mm`. Trying to express fit as a vector term produces results that look right and do not fit, which is the exact failure the product exists to prevent.

Failure it owns. An empty result set. Always fall back to relaxing the fit filter by 10% and labelling the results as such, rather than returning nothing on stage.

P5 – Voice (OMNI)

F One unbroken loop, not three scattered features. Hold the phone at an object for vision, ask the question by speech, identify from live video, reason against the measured geometry, and answer aloud.

The detail. Huawei OMNI Live requires vision, speech, and language together on an edge device. A loop satisfies that. Three separate features do not, and the hard three-modality filter is what thins the field.

P6 – Headset sync

B → D. Phone posts to `/push/{roomId}`, the room Durable Object fans out over SSE, the Quest receives it.

The detail. Bring a travel router, about \$40, and put every device on it. Conference Wi-Fi at peak judging will not carry phone → server → Quest. This is the single largest latency risk and the cheapest to remove.

Failure it owns. Dead air during the handoff. Measure the end-to-end push time on the venue network during the rehearsal block in §9, not at 3 am with the access point to yourself.

Sprint plan, 36 hours

H0 is Friday 18:00. Track lock is H20, Saturday 14:00. Submission is H36, Sunday 06:00. Adjust H0 in this one place if the real schedule differs, because every other hour below is relative to it.

H-4 to H0 – before the clock starts

Four things must be true at H0, and none of them is code.

- ☐ A named Swift owner for component A. Without one, the measurement claim is an estimate and the project becomes a worse Kreativ.
- ☐ `/products.json` verified on the chosen merchants. Ten minutes, and it decides whether P3 exists at all.
- ☐ The travel router is bought and tested.
- ☐ Twenty random objects tested through Stable Fast 3D, so the quality boundary is known.

H0-H4 – integration spike, no features

The goal is a skeleton that moves hardcoded data from end to end. Nobody writes a feature in this block. This is where the project fails if it fails.

#	TASK	OWNER	PROVES
1	Commit the four fixtures from §4	B	Everyone unblocks at once
2	Stub <code>/v1</code> behind X-Stub: 1	B	A, D, F build against a real surface
3	Expo native module returns <code>RoomCapture v1</code>	A	The Expo track is live
4	Fixture room renders in the Quest browser at correct scale	D	P1 transport works
5	Phone → server → Quest push of the fixture GLB	A + B + D	P6 works
6	Baseten returns a mesh, timed on the venue network	C	P2 is not a fantasy

Tasks 3 and 6 run in parallel with 1, 2, 4, and 5. Tasks 1 and 2 come first, because they unblock four people.

H4-H10 – core paths

WORKSTREAM	OWNER	CRITICAL PATH
Real <code>/objects</code> , <code>/uploads</code> , <code>/rooms</code> , SSE	B	Yes
Object scan flow, measured box under 1 s	A	Yes
The scale binding, verified with a tape measure	C	Yes
Room rebuild from real capture, grab and move	D	Demo only
Fit validator: door swing and clearance	E	No
Embedding and indexing on <code>state:"ready"</code>	F	No

Exit condition at H10. A judge-shaped person scans a real object and it appears in the headset at true scale. That is the demo spine. Everything after this is depth.

H10-H16 – depth

WORKSTREAM	OWNER	NOTES
Object library, room list, version history UI	A	The Expo track is graded on this, not on the headset
Version write and diff, /push	B	History is nearly free once layouts are JSON
quality tier, pre-bake the catalog	C	Runs unattended while C works on E
Version scrubber, red violation geometry	D	High demo value per hour
Solver on a discretised grid	E	The unclaimed part of the pitch
Hybrid search, phone voice loop	F	Both differentiators from §1
P3 catalog ingest	whoever is free	Cut first, always

H16-H20 – freeze a candidate

Stop starting new work at H16. Get one complete path working, rehearse it once badly, and write the track submissions. Track lock happens at H20 with a working demo in hand, not a hoped-for one.

H20-H28 – second pass

After the lock, build only what the selected tracks are graded on. If Shopify and Rox were not selected, P3 dies here and its owner moves to E or F. If they were selected, P3 is now on the critical path and everything else yields to it.

H28-H31 – feature freeze at H31

H31 is a hard line. After it, only bug fixes, demo data, and rehearsal. A feature that lands at H33 has never been rehearsed and is more likely to break the demo than to improve it.

H31-H33 – demo rehearsal, venue network, floor full

This catches more failures than any other single activity. Run the full 60 seconds five times with different people holding the phone. Time the push. Rehearse the failure path: what you say when generation is slow, and what you do when it fails.

H33-H36 – buffer and submission

Submit at H34, not H36. Keep two hours of slack for the submission system. Rotate sleep so at least two people are alert for judging.

Critical path versus slack

On the critical path. A's native module, B's /objects plus SSE, C's binding. If any of the three slips, the demo has no spine.

Can slip without killing the demo. E, F, D's scrubber, P3, sun simulation, the shareable link, the phone-AR bookend.

Can slip without killing the pitch. Only D. The headset is a demo device with no sponsor track behind it.

Checkpoints and kill criteria

A kill criterion works only if it is decided before the hour arrives and by a named person. Assign each gate to someone who is not the owner of the work being judged.

GATE	HOURL	TEST	IF IT FAILS
Native module	H4	The Expo module returns valid RoomCapture v1	Drop Expo, ship pure Swift, lose the primary track. Decide at H4, not H6.
Skeleton	H4	Fixture GLB travels phone → server → Quest	Stop feature work. Every person debugs transport until it passes.
Binding	H6	A generated mesh measures correct against a tape measure	Fall back to a textured box at measured dimensions and keep going.
Demo spine	H10	A real object scanned by a stranger reaches the headset at true scale	Cut D entirely and move to the phone-only demo in §10.
Track lock	H20	See below	—
Feature freeze	H31	Nothing merges except fixes	Revert the branch. Do not negotiate this one at H32.

The H20 track lock, Saturday 14:00

This is the hardest checkpoint, because the tracks are selected before the work is finished. Select 5 to 7. Twelve shallow selections read as desperate and dilute every pitch.

Ask three questions at H20, in this order.

1. **Does the fit engine exist?** If not, this is a visualisation project, and visualisation is the lane Kreativ already owns. Select tracks for what exists, and change the pitch that afternoon rather than at the judging table.
2. **Does P3 catalog ingest exist?** Shopify and Rox both hang on it, so they are one bet, not two. Select both or neither.
3. **Does the voice loop run all three modalities?** Huawei OMNI Live is a hard filter. Two of three does not qualify.

The rehearsal block is a checkpoint too

Rehearse on the venue network with the floor full, at H31–H33. Not at 3 am with the access point to yourself. Latency at peak judging is a different number from latency at 3 am, and it is the number the judges will experience.

Rehearse the failure path explicitly. Decide now what the presenter says when generation takes 20 seconds, and what happens when it returns a bad mesh. A rehearsed failure looks like composure. An unrehearsed one looks like a broken project.

Station setup, fixed at H31

- A monitor casting the Quest view. This is not optional. Without it, four bystanders see nothing during the best moment.
- The phone on the table, unlocked, camera session already warm.

- A tape measure on the table. It is the cheapest credibility available, and it converts every claim into something a judge can check.
- Every device on the travel router.

Judge parallelisation

Judges arrive in clumps, so formalise it. Whoever holds the phone is the participant. Everyone else watches the monitor. One judge scans while the others watch the previous result. Generation latency becomes fill time instead of dead air.

The headset is an upsell for a judge who lingers, never a step in the main path. This fixes the latency problem and the throughput problem at once.

Minimum viable demo

The smallest set that still produces a compelling 60 seconds is **A + B + C + one check from E**. D is not in it.

What is in

NEED	COMPONENT	WHY IT CANNOT BE CUT
Scan a judge's own object	A	This is the differentiator. Every competitor is catalog-locked.
Measured box in under 1 s	A	It carries the latency and the credibility at the same moment.
A mesh bound to that measurement	C	Without the binding the scene is decorative and the pitch is false.
Store and serve it	B	Minimal: <code>/objects</code> , <code>/uploads</code> , <code>/generate</code> , <code>/jobs</code> .
One fit violation with a number	E	Door swing only. It is the beat nobody else can do.
A pre-scanned room	A, frozen as a fixture	Captured Friday. A judge does not have to supply a room.

What is out of the minimum

P3 catalog ingest, hybrid search, the solver, the multi-agent loop, version history, sun simulation, the shareable link, in-headset voice, and the entire Quest runtime.

The 60 seconds

1. The room is already loaded and already on the screen.
2. The judge picks up the phone and scans an object they brought.
3. The box snaps on with real numbers: "it is already measured, 31.3 by 22.1 centimetres. The mesh is generating."
4. 10 to 15 seconds of narration while it generates. The thesis line goes here: **every other tool can only show you things that are for sale. This one takes the thing in your hand.**
5. The object drops into the room at true scale.
6. Move it toward the door. The fit engine fires: "that blocks the door swing by 11 centimetres." The arc draws in red.

7. Hand them the tape measure.

That is the protected core. Step 3 and step 6 are the two beats that must never break.

The phone-only fallback

If the Quest fails at 5 am, the demo does not change much. Steps 1 to 7 all run on the phone, with AR placement instead of a headset walkthrough. This is deliberate: there is no AR/VR track in 2026, so the headset earns no sponsor points and must never be a single point of failure.

Keep the headset as the upsell it is. A judge who lingers puts it on. A judge in a hurry sees the whole pitch on the phone.

The two upgrades worth adding first

When the minimum is safe, add these two before anything else, in this order.

1. **Hybrid search**, from F. "Find something that fits the 80 cm gap and matches this wood tone" is the second claim no competitor can make, and it needs only `/search` plus a results list.
2. **The 1:1 headset walkthrough**, from D. It converts a good demo into a memorable one, and it is the moment people tell other people about.

Version history is third. It is cheap, it is a strong visual beat, and it gives the Expo judges something to grade that is not the headset.

Tracks and risks

Select 5 to 7 at H20. Expo is primary. Cloudflare rises to tier 1 under the v2 architecture, because Workers, Vectorize, D1, and R2 now carry the differentiator rather than just hosting it.

TIER	TRACK	WHAT EARNS IT	CONTINGENT ON
1	Expo – Best Mobile Experience	The whole product is an Expo app: rooms, object library, version history, scan flows. Two custom native modules wrapping Apple frameworks. Most Expo entries never leave the JavaScript sandbox.	Component A at H4
1	Cloudflare – Best Agent with a Brain	Workers orchestrate, Vectorize holds the embeddings of your own possessions, D1 holds rooms and versions, R2 holds meshes. The retrieval claim in §1 is literally a Cloudflare feature.	B plus F search
1	Baseten	Image-to-3D cannot run on the phone, so hosted inference is load-bearing. Two latency tiers. Two Baseten people sit on the judging panel, which is the best attention signal available this year.	Nothing. The safest track on the list.
1	Huawei OMNI Live	Vision, speech, and language in one loop on a phone. The three-modality requirement is a hard filter that thins the field. Cloud API access is explicitly permitted.	P5, which is cheap
2	Shopify	Real products with real dimensions at true scale, and the budget re-solve.	P3, the most cuttable piece
2	Rox	The dimension extraction pipeline: conflicting sources, unit validation, aspect-ratio cross-check, confidence scoring, "unverified fit" flagging. Honest handling of ambiguity is in their rubric.	Same as Shopify. One bet, not two.

TIER	TRACK	WHAT EARNS IT	CONTINGENT ON
2	Huawei openJiuwen	The multi-agent loop: scout → fit → style → budget.	F, after search ships
3	ElevenLabs or Gemini	Whichever half of the voice loop OMNI does not cover.	Free

Risks

RISK	SEVERITY	MITIGATION
Kreativ comparison from a judge who has used it	Structural	Answer in one sentence and move: Kreativ places IKEA products, and we place the thing in your hand. Then scan their object. Never argue accuracy.
"AI makes 3D, you look at it in a headset" is a worn lane	Structural	Lead with catalog-free ingest and retrieval, not with visualisation. If those are the centre of the pitch, this weakens a lot.
No AR/VR track exists in 2026	Structural	Accept it. Budget headset hours as demo spend. Keep the phone-only path complete.
10-20 s scan-to-headset latency	High	Instant measured box, narration, judge parallelisation, own router.
No Swift owner	High	Verify before H0. There is no mitigation after H0.
Mesh quality on hard objects	Medium	Test 20 objects Friday. Surface confidence. Steer toward matte solids. A water bottle is the adversarial case.
Merchants block <code>/products.json</code>	Medium	Verify before H0. If blocked, P3 and both tier-2 tracks die together.
Headset handoff eats demo time	Medium	Casting monitor. The headset is an upsell, never the main path.
Judge does not supply a room	Low	They supply the object. Lean on that, because it is the differentiator anyway.
USDZ → GLB conversion	Avoided	JSON transport, rebuilt from primitives. See P1.

Open questions

- Who owns component A? Nothing else in the document matters until this is a name.
- Is the demo room scanned in advance, or scanned live at the booth? A pre-scanned fixture is safer and removes 40 seconds from a 60-second demo.
- Which 15–25 merchants, and are they verified?